DESIGN AND ANALYSIS OF ALGORITHMS

(Practical 2)

SOURCE CODE

```
import heapq

def huffman_encoding(text):

    freq = {}
    for ch in text:
        freq[ch] = freq.get(ch, 0) + 1

    heap = [[weight, [ch, ""]] for ch, weight in freq.items()]
    heapq.heapify(heap)

    while len(heap) > 1:
        lo = heapq.heappop(heap)
        hi = heapq.heappop(heap)
        for pair in lo[1:]:
            pair[1] = "0" + pair[1]
        for pair in hi[1:]:
            pair[1] = "1" + pair[1]
        heapq.heappush(heap, [lo[0] + hi[0]] + lo[1:] + hi[1:])

    huff_codes = dict(sorted(heap[0][1:], key=lambda p: (len(p[-1]), p)))
    encoded_text = "".join(huff_codes[ch] for ch in text)

    return huff_codes, encoded_text

def huffman_decoding(encoded_text, codes):

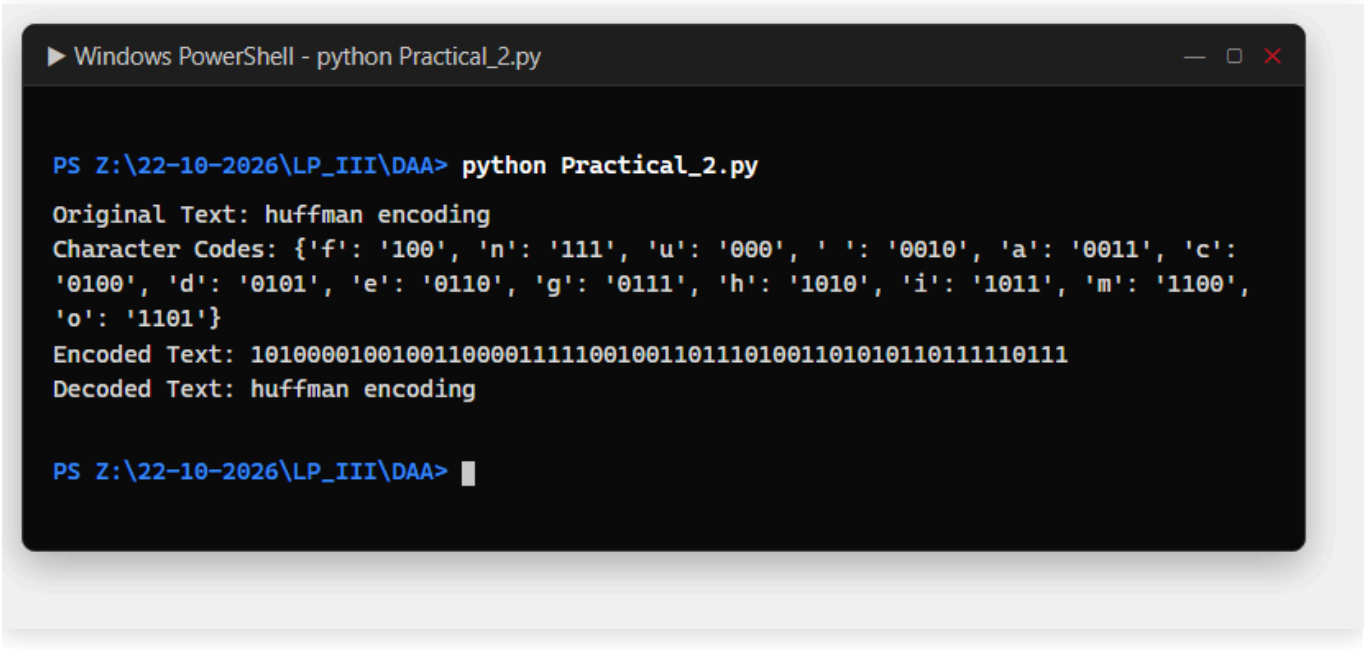
    reverse_codes = {v: k for k, v in codes.items()}
    decoded_text = ""
    current = ""
    for bit in encoded_text:
        current += bit
        if current in reverse_codes:
            decoded_text += reverse_codes[current]
            current = ""
    return decoded_text

text = "huffman encoding"
codes, encoded = huffman_encoding(text)
```

```
decoded = huffman_decoding(encoded, codes)

print("Original Text:", text)
print("Character Codes:", codes)
print("Encoded Text:", encoded)
print("Decoded Text:", decoded)
```

OUTPUT


Windows PowerShell - python Practical_2.py
PS Z:\22-10-2026\LP_III\DAA> python Practical_2.py
Original Text: huffman encoding
Character Codes: {'f': '100', 'n': '111', 'u': '000', ' ': '0010', 'a': '0011', 'c': '0100', 'd': '0101', 'e': '0110', 'g': '0111', 'h': '1010', 'i': '1011', 'm': '1100', 'o': '1101'}
Encoded Text: 10100001001001100001111100100110111010011010101101111110111
Decoded Text: huffman encoding
PS Z:\22-10-2026\LP_III\DAA>